Reviewer Pack — Real Radical Rank Bounds SOS Degree for Max-CUT

Entry f54522e2e780 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-09 04:05:16 UTC

Real Radical Rank Bounds SOS Degree for Max-CUT

Entry ID: `f54522e2e780` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-09 04:05:16 UTC

1. Conjecture

Field A (mathematical branch): Real Algebraic Geometry **Field B** (complexity object): SOS Degree for Max-CUT

Statement:

> For a random Max-CUT instance on n variables, the minimal SOS degree d(n) required to approximate the maximum cut satisfies d(n) ≥ log2(rank(ℝ-radical(I))), where I is the ideal of polynomials vanishing on the feasible region of the instance.

Rationale (proposer's reasoning):

> The real radical of an ideal captures the semialgebraic structure of the feasible region. Its rank, as a measure of algebraic complexity, may correlate with the SOS hierarchy's depth needed to certify optimality, offering a geometric lens on convex relaxation trade-offs.

Taxonomy category: `SOS_DEGREE` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `5e26d181c3fe59ea`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.95	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_max_cut_instance(n):
        return [random.choice([-1, 1]) for _ in range(n)]

    def matrix_multiply(A, B):
        n = len(A)
        C = [[0] * n for _ in range(n)]
        for i in range(n):
            for j in range(n):
                for k in range(n):
                    C[i][j] += A[i][k] * B[k][j]
        return C

    def gaussian_elimination(A, b):
        n = len(A)
        M = [A[i] + [b[i]] for i in range(n)]
        for i in range(n):
            max_row = max(range(i, n), key=lambda r: abs(M[r][i]))
            M[i], M[max_row] = M[max_row], M[i]
            factor = M[i][i]
            for j in range(i, n + 1):
                M[i][j] /= factor
            for k in range(n):
                if k != i:
                    factor = M[k][i]
                    for j in range(i, n + 1):
                        M[k][j] -= factor * M[i][j]
        return [row[-1] for row in M]

    def determinant(A):
        n = len(A)
        det = 0
        if n == 1:
            return A[0][0]
        elif n == 2:
            return A[0][0] * A[1][1] - A[0][1] * A[1][0]
        else:
            for j in range(n):
                det += (-1) ** j * A[0][j] * determinant([row[:j] + row[j+1:] for row in A[1:]])
        return det
```

```
    def inverse(A):
        n = len(A)
        det_A = determinant(A)
        if det_A == 0:
            raise ValueError("Modular inverse does not exist")
        adjoint = [[0] * n for _ in range(n)]
        for i in range(n):
            for j in range(n):
                minor = [row[:j] + row[j+1:] for row in A[:i] + A[i+1:]]
                cofactor = determinant(minor) * ((-1) ** (i + j))
                adjoint[j][i] = cofactor
        return [[adjoint[i][j] / det_A for j in range(n)] for i in range(n)]

    def real_radical_rank(I):
        n = len(I)
        A = [[I[i][j] * I[k][l] for l in range(n)] for k in range(n) for j in range(n)]
        A = [sum(A[j*n+i] for j in range(n)) for i in range(n)]
        A = [[A[i*n+j] for j in range(n)] for i in range(n)]
        det_A = determinant(A)
        return int(math.log2(det_A))

    def sos_degree(instance):
        n = len(instance)
        A = [[0] * (n + 1) for _ in range(n + 1)]
        for i in range(n):
            for j in range(n):
                A[i][j] = instance[i] * instance[j]
            A[i][n] = -instance[i]
        A[n][n] = 1
        b = [0] * (n + 1)
        x = gaussian_elimination(A, b)
        return n

    def max_cut_approximation(instance):
        n = len(instance)
        cut_value = sum(abs(x) for x in instance) / 2
        return cut_value

    n = 40
    instance = generate_max_cut_instance(n)
    rank_I = real_radical_rank([instance])
    d_n = sos_degree(instance)
    conjecture_holds = d_n >= math.log2(rank_I)

    return {
        "metric_name": "real_radical_rank",
        "metric_value": rank_I,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": "" if conjecture_holds else f"instance={instance}"
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2**i + 3 for i in range(5, 8)]
    results = []
```

```
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6b41a151.py", line 118, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6b41a151.py", line 101, in run_trial
    rank_I = real_radical_rank([instance])
             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6b41a151.py", line 76, in real_radical_
    A = [[I[i][j] * I[k][l] for l in range(n)] for k in range(n) for j in range(n)]
          ^
NameError: name 'i' is not defined. Did you mean: 'I'?
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> Test crashed due to NameError, preventing data collection to evaluate conjecture | next: Fix the NameError in real_radical_rank function by defining variables properly

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	86429
2	preregistration	ollama_remote	qwen3:8b	0	0	24395
3	novelty	ollama_remote	qwen3:8b	0	0	20701
4	novelty	ollama_remote	qwen3:8b	0	0	15236
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	24748
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13917
7	judge	ollama_remote	qwen3:8b	0	0	15497

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 200924 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in `research/audit/f54522e2e780.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/f54522e2e780.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/f54522e2e780.tar.gz` (if generated)